



Web Security Code Review + JWT

Louis Nyffenegger
louis@pentesterlab.com





ABOUT ME:

- Founder and CEO of PentesterLab
- Ex: Pentester, Code Reviewer, AppSec Engineer





- Online Platform to Learn Code Review and Web Hacking / Web Penetration Testing
- Online Live Training Sessions on Web Security Code Review



This WorkShop

- Introduction
- Routing
- Patterns
- CVE Analysis
- Introduction to JWT
- Algorithm Conclusion
- Conclusion



INTRODUCTION

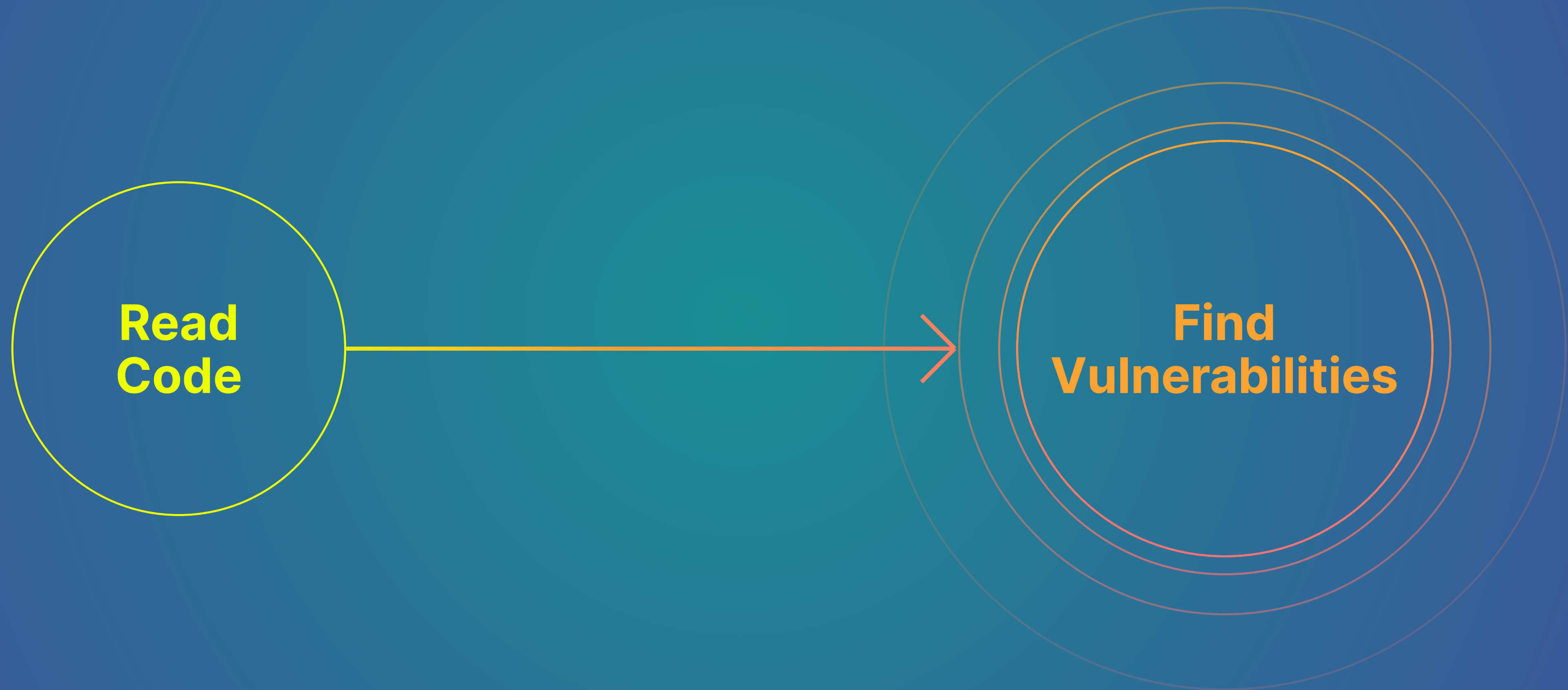


Security Code Review is in demand

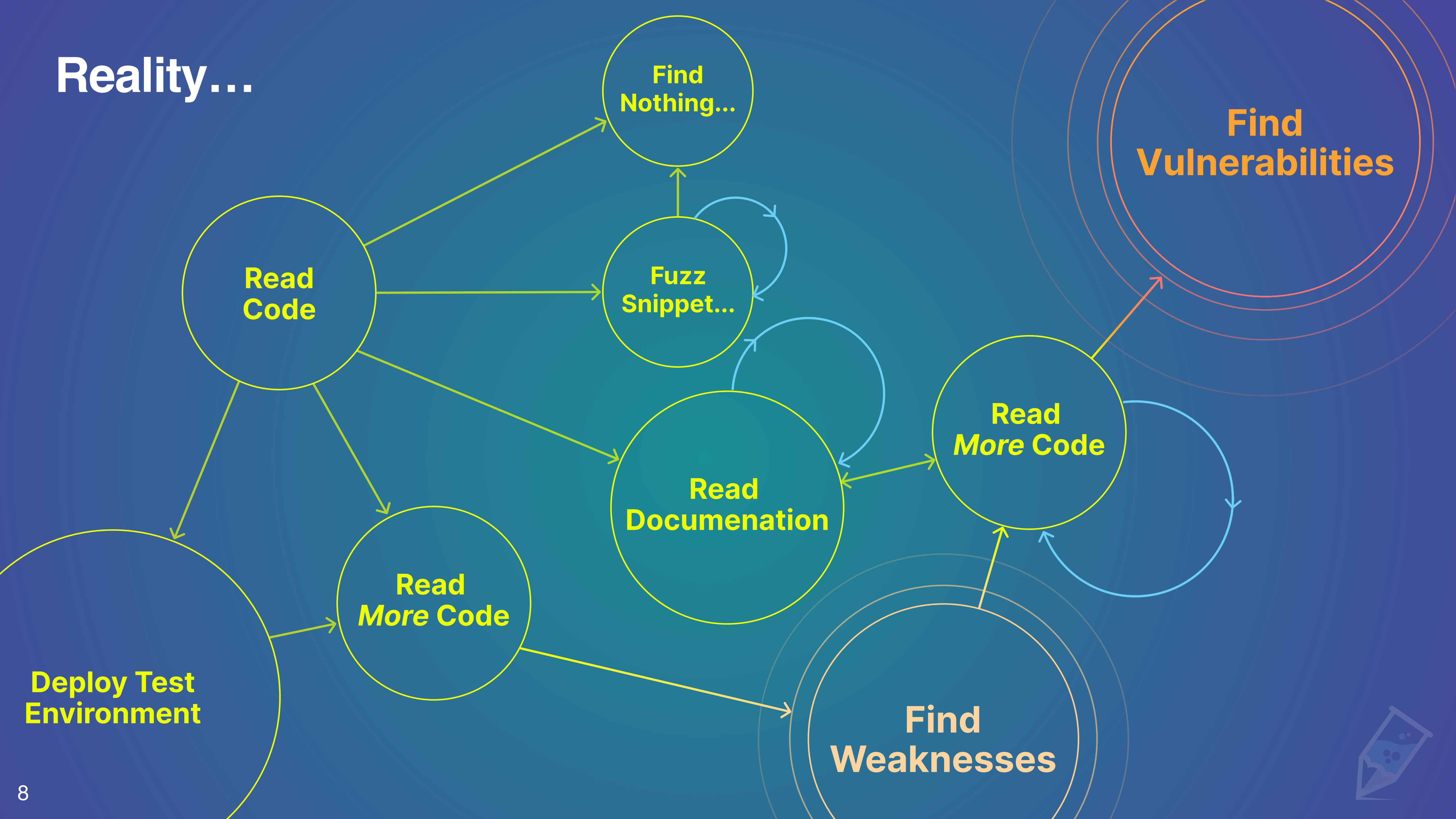
- Ability to find complex bugs
- Ability to find bugs that scanners can't find
- Ability to review changes prior to deployment (Agile, AppSec)
- Ability to find new classes of vulnerabilities
- Powerful skill for:
 - Developers
 - Penetration Testers
 - Security Engineers
 - Vulnerability Researchers / Exploit writers
 - QA/Test Engineer



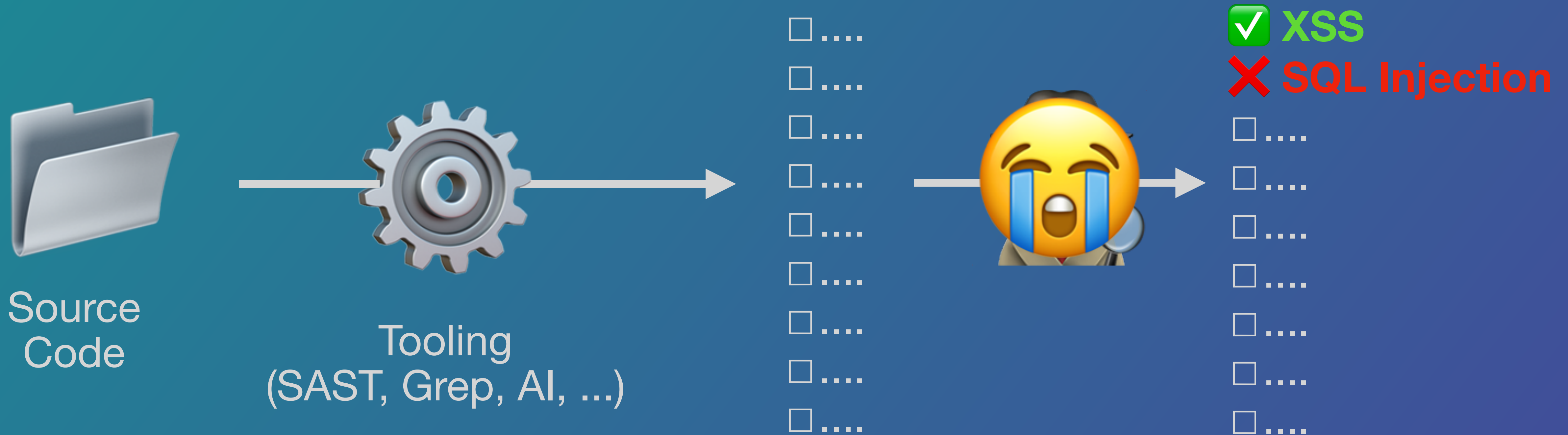
Expectations...

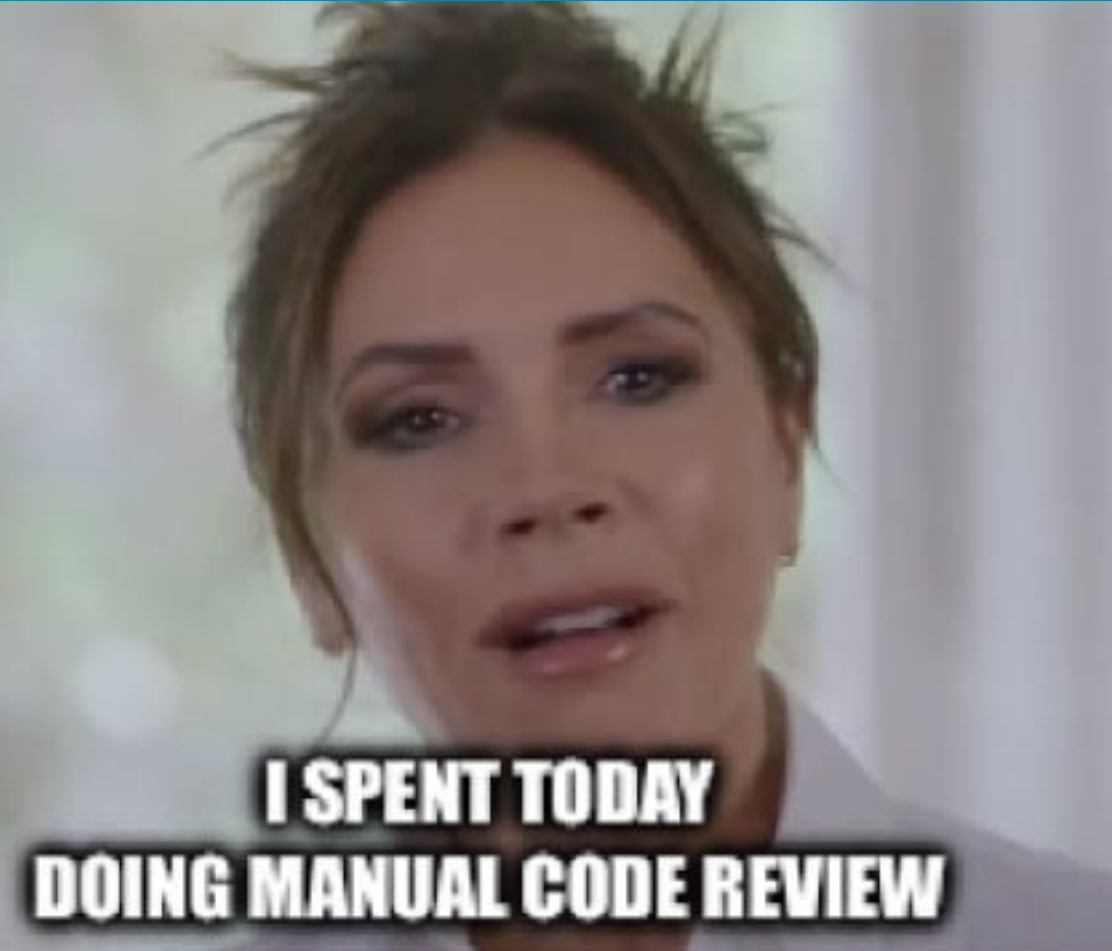


Reality...

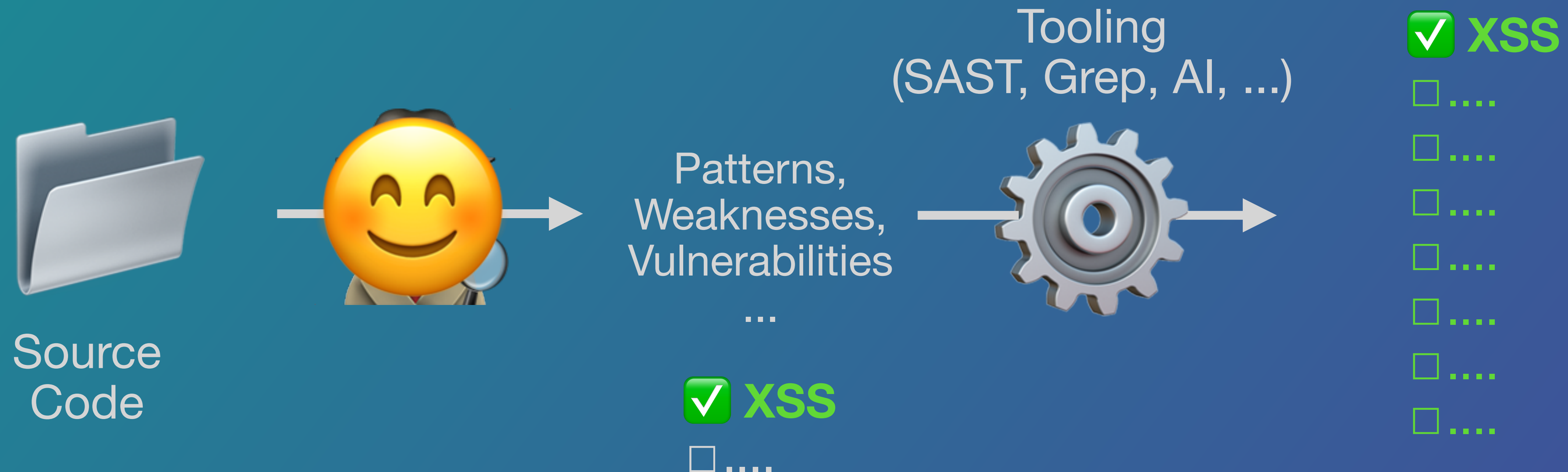


Security Code Review...





Security Code Review...



One of the main advantages of this approach is that it helps identify “unknown unknowns”—issues that automated tools may overlook.



Should you know the language?

A good rule of thumb is that you need to know things that developers don't know:

- Something about a format used?
- A way to bypass a filter?
- Something about threat modelling?
- Something about the language?

The more things you know that the developers don't, the more likely you are to find vulnerabilities



Should you know how to write code?

- It definitely helps!
- You don't need to be a "real" developer but knowing how applications are developed will speed up your work
- The more code you write, the more likely you are to guess:
 - What mistakes developers will make?
 - What shortcuts developers will take?
- The less you know, the more patient you will have to be



Threat modeling

- Key component of code review
- If you don't know what can go wrong, you don't know what to check for
- Knowing about common bug classes for each type of feature or application is key
- Threat modeling gives "direction" to your review

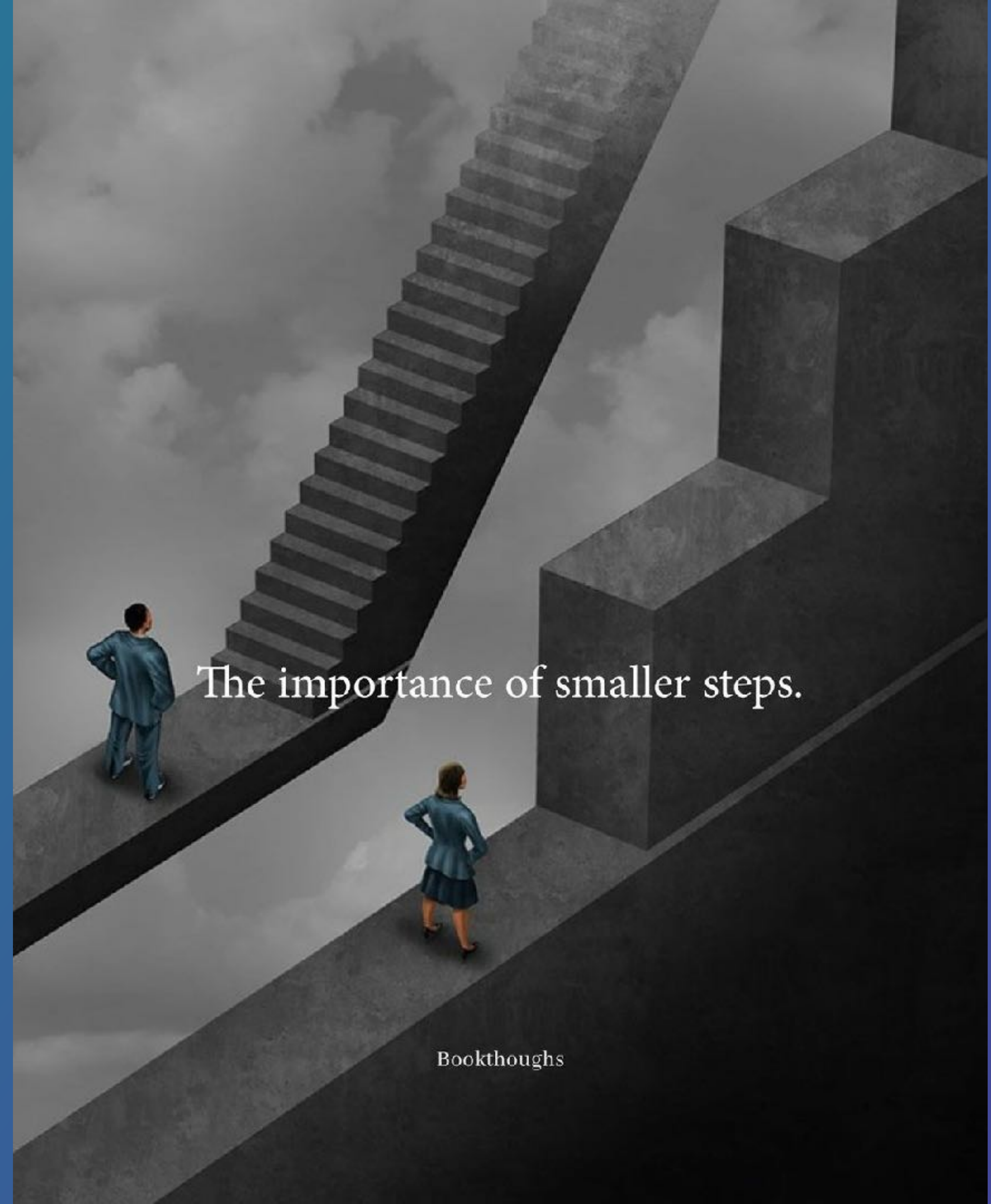


Threat modeling: How to learn?

- There are many methodologies for threat modeling
- For web security code reviews, your best options are to:
 - Read pentest reports
 - Read bug bounty findings and write-ups
 - Follow research presented at conferences
 - Analyze CVE



Picking your targets to learn...



Bookthoughts

Picking your targets to learn...

- You need to find targets that are not too easy
- You need to find targets that are not too hard
- You need to find targets that allow you to grow
- You need to find targets to build resilience



Picking your targets to learn...

1. Snippets
2. Diff from known/public vulnerabilities/CVE
3. Small or simple Libraries
4. Bigger or more complex Libraries
5. Small Applications
6. Larger Applications
7. Hard Targets



Defining Success in Security Code Review

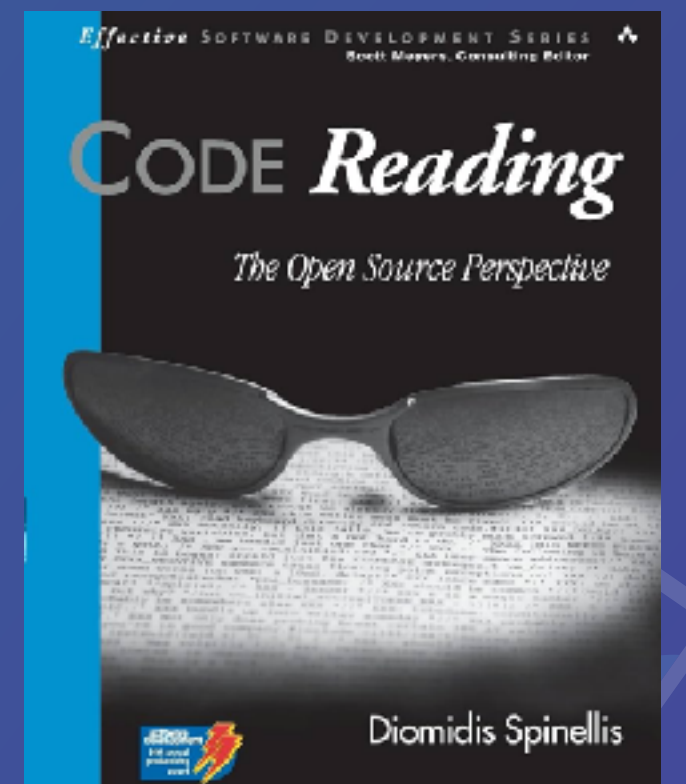
- You don't want to base your success based on the number of vulnerabilities you find or the impact of the vulnerabilities you find (especially when learning)
- Success should be based on:
 - Your progression in understanding a codebase
 - Learning ways a check or filter is implemented
 - Finding small weaknesses or potential improvements
 - Understanding complex patterns
 - Discovering new patterns (with and without security implications)



READING CODE



"An hour of code reading can save you a minute of reading the documentation"



READING THE CODE



Notice:

- Things that are unusual
- When people reinvent the wheel
- Sketchy code
- Complexity
- Unchecked return values
- Checks:
 - What are they trying to prevent?
 - Are they preventing it properly?
 - Is there something else they should take care of but they don't?



Reading code

- Every time you encounter a new function or method:
 - Read the documentation
 - Look for potential security improvements and issues
 - "Fuzz" it (REPL or docker)
 - Keep notes

**Bonus point for doing this over multiple versions of the same method/function...*



PATTERNS...



Patterns

When the sage points at the moon, the fool looks at the finger.

- **A lot of issues in security are completely independent of the programming language**
- **In this section, we are going to explore patterns with implementation in multiple programming languages**
- **Make sure you focus on the pattern**



Filter -> Modify -> Use

- The code does three things:
 1. Filters for malicious values
 2. Modifies the value
 3. Uses the value

CAN WE REINTRODUCE SOME
OF THE FILTERED VALUES BACK
USING THE MODIFICATION?



Filter -> Modify -> Use



```
static String validateFileName( String filename )
    throws Exception {
    if( filename == null || filename.trim().isEmpty() ) {
        throw new Exception("Empty File Name");
    }
    final String[] splitpath = filename.split( "[/\\\\\\]" );
    filename = splitpath[splitpath.length-1];

    filename = filename.trim();

    // If file name ends with .jsp or .jspx,
    //the user is being naughty!
    if( filename.toLowerCase().endsWith( ".jsp" ) ||
        filename.toLowerCase().endsWith( ".jspx" ) ) {
        throw new Exception("Dangerous extension");
    }

    // Remove any characters that might be a problem.
    return filename.replaceAll("( [?#'\\";] )", "" );
}
```



Filter -> Modify -> Use



```
static String validateFileName( String filename )
    throws Exception {
    if( filename == null || filename.trim().isEmpty() ) {
        throw new Exception("Empty File Name");
    }
    final String[] splitpath = filename.split( "[/\\\\\\]" );
    filename = splitpath[splitpath.length-1];

    filename = filename.trim();

    // If file name ends with .jsp or .jspx,
    //the user is being naughty!
    if( filename.toLowerCase().endsWith( ".jsp" ) ||
        filename.toLowerCase().endsWith( ".jspx" ) ) {
        throw new Exception("Dangerous extension");
    }

    // Remove any characters that might be a problem.
    return filename.replaceAll("( [?#'\\";] )", "" );
}
```

1



Filter -> Modify -> Use



```
static String validateFileName( String filename )
    throws Exception {
    if( filename == null || filename.trim().isEmpty() ) {
        throw new Exception("Empty File Name");
    }
    final String[] splitpath = filename.split( "[/\\\\\\]" );
    filename = splitpath[splitpath.length-1];

    filename = filename.trim();

    // If file name ends with .jsp or .jspx,
    //the user is being naughty!
    if( filename.toLowerCase().endsWith( ".jsp" ) ||
        filename.toLowerCase().endsWith( ".jspx" ) ) {
        throw new Exception("Dangerous extension");
    }

    // Remove any characters that might be a problem.
    return filename.replaceAll("( [?#'\\";] )", "" );
}
```

1

2



Filter -> Modify -> Use



```
static String validateFileName( String filename )
    throws Exception {
    if( filename == null || filename.trim().isEmpty() ) {
        throw new Exception("Empty File Name");
    }
    final String[] splitpath = filename.split( "[/\\\\\\]" );
    filename = splitpath[splitpath.length-1];

    filename = filename.trim();

    // If file name ends with .jsp or .jspx,
    //the user is being naughty!
    if( filename.toLowerCase().endsWith( ".jsp" ) ||
        filename.toLowerCase().endsWith( ".jspx" ) ) {
        throw new Exception("Dangerous extension");
    }

    // Remove any characters that might be a problem.
    return filename.replaceAll("( [?#'\\";] )", "" );
}
```

1

2

hack.jsp#



Filter -> Modify -> Use



```
public static String cleanName(String name) {  
    return Normalizer.normalize(  
        HtmlUtil.encode(  
            name.replace(" ", "_")  
                .replace("&", "")  
                .replace("(", "")  
                .replace(")", "")  
                .replace(",", "")  
                .replace("+", "_"), HtmlUtil.ENCODING_TEXT), Normalizer.Form.NFC);  
}
```



Filter -> Modify -> Use



```
public static String cleanName(String name) {  
    return Normalizer.normalize(  
        HtmlUtil.encode(  
            name.replace(" ", "_")  
                .replace("&", "")  
                .replace("(", "")  
                .replace(")", "")  
                .replace(",", "")  
                .replace("+", "_"), HtmlUtil.ENCODING_TEXT), Normalizer.Form.NFC);  
}
```

Filter



Filter -> Modify -> Use



```
public static String cleanName(String name) {  
    return Normalizer.normalize(  
        HtmlUtil.encode(  
            name.replace(" ", "_")  
                .replace("&", "")  
                .replace("(", "")  
                .replace(")", "")  
                .replace(",", "")  
                .replace("+", "_"), HtmlUtil.ENCODING_TEXT), Normalizer.Form.NFC);  
}
```

Modify!



Filter -> Modify -> Use



```
public static String cleanName(String name) {  
    return Normalizer.normalize(  
        HtmlUtil.encode(  
            name.replace(" ", "_")  
                .replace("&", "")  
                .replace("(", "")  
                .replace(")", "")  
                .replace(",", "")  
                .replace("+", "_"), HtmlUtil.ENCODING_TEXT), Normalizer.Form.NFC);  
}
```

The modification may reintroduce things the code filtered...



Filter -> Modify -> Use



```
- return java.text.Normalizer.normalize(
-     HtmlUtil.encode(
-         name.replace(" ", "_")
-         .replace("&", "&")
-         .replace("(", "(")
-         .replace(")", ")")
-         .replace(",", ",")
-         .replace("+", "+")
-         .replace(".", "."), HtmlUtil.ENCODE_TEXT), Normalizer.Form.NFC);
+ return HtmlUtil.encode(Normalizer.normalize(name, Normalizer.Form.NFC)
+     .replace(" ", "_")
+     .replace("&", "&")
+     .replace("(", "(")
+     .replace(")", ")")
+     .replace(",", ",")
+     .replace("+", "+"),
+     HtmlUtil.ENCODE_TEXT);
```

Normalize then filter/escape

Matching is hard

Ends with, contains, starts with...

- When matching strings without using a Regular Expression, a lot of people get confused on what they are trying to achieve.
- "ends with", "contains", "starts with" may feel like they work similarly for the happy path but they rarely do in reality.



Matching is hard

Ends with, contains, starts with...



```
isLibCurlDomain := strings.Contains(u.UserEmail, "@libcurl.so")
```



Matching is hard

Ends with, contains, starts with...



```
isLibCurlDomain := strings.Contains(u.UserEmail, "@libcurl.so")
```

louis@libcurl.so.pentesterlab.com



Not matching the correct value...

Ends with, contains, starts with...



```
func authMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        tokenString := r.Header.Get("Authorization")

        claims := &Claims{}

        token, err := jwt.ParseWithClaims(tokenString, claims, func(token *jwt.Token) (interface{}, error) {
            return jwtKey, nil
        })

        if err != nil {
            http.Error(w, "Invalid token", http.StatusUnauthorized)
            return
        }

        if !token.Valid {
            http.Error(w, "Invalid token", http.StatusUnauthorized)
            return
        }

        if !strings.Contains(r.URL.String(), "health") && claims.Username != "admin" {
            http.Error(w, "You don't have access to the key", http.StatusUnauthorized)
            return
        }

        next.ServeHTTP(w, r)
    })
}
```



Not matching the correct value...

Ends with, contains, starts with...



```
func authMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        tokenString := r.Header.Get("Authorization")

        claims := &Claims{}

        token, err := jwt.ParseWithClaims(tokenString, claims, func(token *jwt.Token) (interface{}, error) {
            return jwtKey, nil
        })

        if err != nil {
            http.Error(w, "Invalid token", http.StatusUnauthorized)
            return
        }

        if !token.Valid {
            http.Error(w, "Invalid token", http.StatusUnauthorized)
            return
        }

        if !strings.Contains(r.URL.String(), "health") && claims.Username != "admin" {
            http.Error(w, "You don't have access to the key", http.StatusUnauthorized)
            return
        }

        next.ServeHTTP(w, r)
    })
}
```



Not matching the correct value...

Ends with, contains, starts with...



```
func authMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        tokenString := r.Header.Get("Authorization")

        claims := &Claims{}

        token, err := jwt.ParseWithClaims(tokenString, claims, func(token *jwt.Token) (interface{}, error) {
            return jwtKey, nil
        })

        if err != nil {
            http.Error(w, "Invalid token", http.StatusUnauthorized)
            return
        }

        if !token.Valid {
            http.Error(w, "Invalid token", http.StatusUnauthorized)
            return
        }
        if !strings.Contains(r.URL.String(), "health") && claims.Username != "admin" {
            http.Error(w, "You don't have access to the key", http.StatusUnauthorized)
            return
        }
        next.ServeHTTP(w, r)
    })
}
```

r.URL.String() vs r.URL.Path



Reinventing the wheel!

- Never a good idea (always a bad idea when dealing with crypto)
- For most common operations, programming languages provide built-in functions or methods, such as:
 - String manipulations: uppercase, lowercase, split, cut
 - File manipulations: getting the file extension from a filename, extracting the filename from a path, etc.
- When developers write their own versions, they're likely to overlook odd edge cases that the built-in functions already handle
- Compare the built-in source code with the code written by the developers!



Play Session Injection



```
void save() {  
    [...]  
    try {  
        StringBuilder session = new StringBuilder();  
        for (String key : data.keySet()) {  
            session.append("\u0000");  
            session.append(key);  
            session.append(":");  
            session.append(data.get(key));  
            session.append("\u0000");  
        }  
        String sessionData =  
            URLEncoder.encode(session.toString(), "utf-8");  
        String sign = Crypto.sign(sessionData,  
                                   Play.secretKey.getBytes());  
    }  
}
```



Play Session Injection



```
void save() {  
    [...]  
    try {  
        StringBuilder session = new StringBuilder();  
        for (String key : data.keySet()) {  
            session.append("\u0000");  
            session.append(key);  
            session.append(":");  
            session.append(data.get(key));  
            session.append("\u0000");  
        }  
        String sessionData =  
            URLEncoder.encode(session.toString(), "utf-8");  
        String sign = Crypto.sign(sessionData,  
                                   Play.secretKey.getBytes());  
    }  
}
```



**They are
reinventing a
serialiser**



Play Session Injection



```
void save() {  
    [...]  
    try {  
        StringBuilder session = new StringBuilder();  
        for (String key : data.keySet()) {  
            session.append("\u0000");  
            session.append(key);  
            session.append(":");  
            session.append(data.get(key));  
            session.append("\u0000");  
        }  
        String sessionData =
```

Session: {"key1": "value1", "key2": "value2"}

becomes "\x00key1:value1\x00\x00key2:value2\x00"



Play Session Injection



```
void save() {  
    [...]  
    try {  
        StringBuilder session = new StringBuilder();  
        for (String key : data.keySet()) {  
            session.append("\u0000");  
            session.append(key);  
            session.append(":");  
            session.append(data.get(key));  
            session.append("\u0000");  
        }  
        String sessionData =
```

Session: {"username": "louis", "email": "louis@pentesterlab.com"}

becomes "\x00username:louis\x00\x00email:louis@pentesterlab.com\x00"



Play Session Injection



```
public static class Session {
    static Pattern sessionParser =
        Pattern.compile("\\u0000([^:]*):([^\u0000]*)\u0000");

    [...]

    String value = cookie.value;
    int firstDashIndex = value.indexOf("-");
    if(firstDashIndex > -1) {
        String sign = value.substring(0, firstDashIndex);
        String data = value.substring(firstDashIndex + 1);
        if (sign.equals(Crypto.sign(data,
                                   Play.secretKey.getBytes())) {
            String sessionData = URLDecoder.decode(data, "utf-8");
            Matcher matcher = sessionParser.matcher(sessionData);
            while (matcher.find()) {
                session.put(matcher.group(1), matcher.group(2));
            }
        }
    }
}
```



Play Session Injection

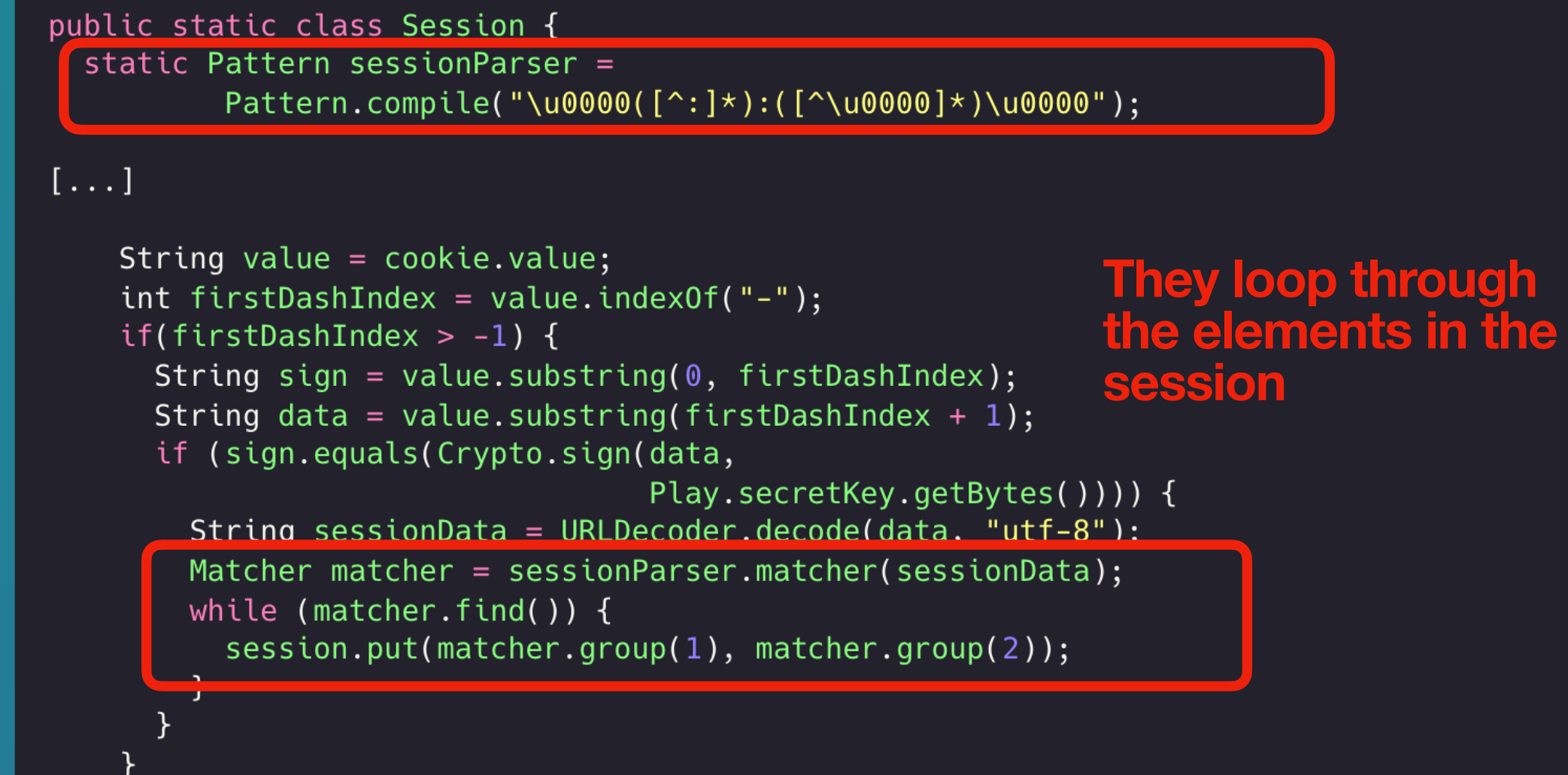


```
public static class Session {  
    static Pattern sessionParser =  
        Pattern.compile("\\u0000([^:]*):(^[\\u0000]*)\\u0000");  
  
    [...]  
  
    String value = cookie.value;  
    int firstDashIndex = value.indexOf("-");  
    if (firstDashIndex > -1) {  
        String sign = value.substring(0, firstDashIndex);  
        String data = value.substring(firstDashIndex + 1);  
        if (sign.equals(Crypto.sign(data,  
                                    Play.secretKey.getBytes())) {  
            String sessionData = URLDecoder.decode(data, "utf-8");  
            Matcher matcher = sessionParser.matcher(sessionData);  
            while (matcher.find()) {  
                session.put(matcher.group(1), matcher.group(2));  
            }  
        }  
    }  
}
```

**They loop
through the
elements in the
session**



Play Session Injection



```
public static class Session {  
    static Pattern sessionParser =  
        Pattern.compile("\\u0000(?:[^\u0000]*)\\u0000");  
    [...]  
  
    String value = cookie.value;  
    int firstDashIndex = value.indexOf("-");  
    if (firstDashIndex > -1) {  
        String sign = value.substring(0, firstDashIndex);  
        String data = value.substring(firstDashIndex + 1);  
        if (sign.equals(Crypto.sign(data,  
            Play.secretKey.getBytes())) {  
            String sessionData = URLDecoder.decode(data, "utf-8");  
            Matcher matcher = sessionParser.matcher(sessionData);  
            while (matcher.find()) {  
                session.put(matcher.group(1), matcher.group(2));  
            }  
        }  
    }  
}
```

They loop through the elements in the session

"\x00key1:value1\x00\x00key2:value2\x00" becomes:

"\x00key1:value1\x00" => session.put("key1" , "value1")

"\x00key2:value2\x00" => session.put("key2" , "value2")



Play Session Injection



```
public void put(String key, String value) {  
    if (key.contains(":")) {  
        throw new IllegalArgumentException(  
            "Character ':' is invalid in a session key.");  
    }  
    [...]   
    if (value == null) {  
        data.remove(key);  
    } else {  
        data.put(key, value);  
    }  
}
```



Play Session Injection



```
public void put(String key, String value) {  
    if (key.contains(":")) {  
        throw new IllegalArgumentException(  
            "Character ':' is invalid in a session key.");  
    }  
    [...]  
    if (value == null) {  
        data.remove(key);  
    } else {  
        data.put(key, value);  
    }  
}
```



**No checks to
prevent separators
(':' or NULL BYTE) in
the value**



Play Session Injection



```
public void put(String key, String value) {  
    if (key.contains(":")) {  
        throw new IllegalArgumentException(  
            "Character ':' is invalid in a session key.");  
    }  
    [...]   
    if (value == null) {  
        data.remove(key);  
    } else {  
        data.put(key, value);  
    }  
}
```

As a client, we most likely only have access to the value.



Play Session Injection



```
public static class Session {
    static Pattern sessionParser =
        Pattern.compile("\\u0000([^:]*):(^[\\u0000]*)\\u0000");

    [...]

    String value = cookie.value;
    int firstDashIndex = value.indexOf("-");
    if(firstDashIndex > -1) {
        String sign = value.substring(0, firstDashIndex);
        String data = value.substring(firstDashIndex + 1);
        if (sign.equals(Crypto.sign(data,
                                   Play.secretKey.getBytes())) {
            String sessionData = URLDecoder.decode(data, "utf-8");
            Matcher matcher = sessionParser.matcher(sessionData);
            while (matcher.find()) {
                session.put(matcher.group(1), matcher.group(2));
            }
        }
    }
}
```

username=[USER-CONTROLLED]



Play Session Injection



```
public static class Session {
    static Pattern sessionParser =
        Pattern.compile("\\u0000([^:]*):(^[\\u0000]*)\\u0000");

    [...]

    String value = cookie.value;
    int firstDashIndex = value.indexOf("-");
    if(firstDashIndex > -1) {
        String sign = value.substring(0, firstDashIndex);
        String data = value.substring(firstDashIndex + 1);
        if (sign.equals(Crypto.sign(data,
                                   Play.secretKey.getBytes())) {
            String sessionData = URLDecoder.decode(data, "utf-8");
            Matcher matcher = sessionParser.matcher(sessionData);
            while (matcher.find()) {
                session.put(matcher.group(1), matcher.group(2));
            }
        }
    }
}
```

username=louis



Play Session Injection



```
public static class Session {
    static Pattern sessionParser =
        Pattern.compile("\\u0000([^:]*):([^\u0000]*)\u0000");

    [...]

    String value = cookie.value;
    int firstDashIndex = value.indexOf("-");
    if(firstDashIndex > -1) {
        String sign = value.substring(0, firstDashIndex);
        String data = value.substring(firstDashIndex + 1);
        if (sign.equals(Crypto.sign(data,
                                   Play.secretKey.getBytes())) {
            String sessionData = URLDecoder.decode(data, "utf-8");
            Matcher matcher = sessionParser.matcher(sessionData);
            while (matcher.find()) {
                session.put(matcher.group(1), matcher.group(2));
            }
        }
    }
}
```

username=louis => session.put("username", "louis")



Play Session Injection



```
public static class Session {
    static Pattern sessionParser =
        Pattern.compile("\\u0000([^:]*):(^[\\u0000]*)\\u0000");

    [...]

    String value = cookie.value;
    int firstDashIndex = value.indexOf("-");
    if(firstDashIndex > -1) {
        String sign = value.substring(0, firstDashIndex);
        String data = value.substring(firstDashIndex + 1);
        if (sign.equals(Crypto.sign(data,
                                   Play.secretKey.getBytes())) {
            String sessionData = URLDecoder.decode(data, "utf-8");
            Matcher matcher = sessionParser.matcher(sessionData);
            while (matcher.find()) {
                session.put(matcher.group(1), matcher.group(2));
            }
        }
    }
}
```

username=louis => session.put("username", "louis")
=> "\\x00username:louis\\x00"



Play Session Injection



```
public static class Session {
    static Pattern sessionParser =
        Pattern.compile("\\u0000([^:]*):(^[\\u0000]*)\\u0000");

    [...]

    String value = cookie.value;
    int firstDashIndex = value.indexOf("-");
    if(firstDashIndex > -1) {
        String sign = value.substring(0, firstDashIndex);
        String data = value.substring(firstDashIndex + 1);
        if (sign.equals(Crypto.sign(data,
                                   Play.secretKey.getBytes())) {
            String sessionData = URLDecoder.decode(data, "utf-8");
            Matcher matcher = sessionParser.matcher(sessionData);
            while (matcher.find()) {
                session.put(matcher.group(1), matcher.group(2));
            }
        }
    }
}
```

username=louis\x00\x00username:admin



Play Session Injection



```
public static class Session {
    static Pattern sessionParser =
        Pattern.compile("\u0000(?:[^\u0000]*)\u0000");

    [...]

    String value = cookie.value;
    int firstDashIndex = value.indexOf("-");
    if (firstDashIndex > -1) {
        String sign = value.substring(0, firstDashIndex);
        String data = value.substring(firstDashIndex + 1);
        if (sign.equals(Crypto.sign(data,
            Play.secretKey.getBytes()))) {
            String sessionData = URLDecoder.decode(data, "utf-8");
            Matcher matcher = sessionParser.matcher(sessionData);
            while (matcher.find()) {
                session.put(matcher.group(1), matcher.group(2));
            }
        }
    }
}
```

username=louis\x00\x00username:admin

=> session.put("username", "louis\x00\x00username:admin")



Play Session Injection



```
public static class Session {
    static Pattern sessionParser =
        Pattern.compile("\u0000(?:[^\u0000]*)\u0000");

    [...]

    String value = cookie.value;
    int firstDashIndex = value.indexOf("-");
    if (firstDashIndex > -1) {
        String sign = value.substring(0, firstDashIndex);
        String data = value.substring(firstDashIndex + 1);
        if (sign.equals(Crypto.sign(data,
            Play.secretKey.getBytes()))) {
            String sessionData = URLDecoder.decode(data, "utf-8");
            Matcher matcher = sessionParser.matcher(sessionData);
            while (matcher.find()) {
                session.put(matcher.group(1), matcher.group(2));
            }
        }
    }
}
```

username=louis\x00\x00username:admin

=> session.put("username", "louis\x00\x00username:admin")

=> \x00username:louis\x00\x00username:admin\x00



Play Session Injection



```
public static class Session {
    static Pattern sessionParser =
        Pattern.compile("\u0000(?:[^\u0000]*)\u0000");

    [...]

    String value = cookie.value;
    int firstDashIndex = value.indexOf("-");
    if (firstDashIndex > -1) {
        String sign = value.substring(0, firstDashIndex);
        String data = value.substring(firstDashIndex + 1);
        if (sign.equals(Crypto.sign(data,
            Play.secretKey.getBytes())) {
            String sessionData = URLDecoder.decode(data, "utf-8");
            Matcher matcher = sessionParser.matcher(sessionData);
            while (matcher.find()) {
                session.put(matcher.group(1), matcher.group(2));
            }
        }
    }
}
```

username=louis\x00\x00username:admin

=> session.put("username", "louis\x00\x00username:admin")

=> \x00username:louis\x00\x00username:admin\x00



Play Session Injection



```
public static class Session {  
    static Pattern sessionParser =  
        Pattern.compile("\\u0000(?:[^\u0000]*)\\u0000");  
    [...]  
  
    String value = cookie.value;  
    int firstDashIndex = value.indexOf("-");  
    if (firstDashIndex > -1) {  
        String sign = value.substring(0, firstDashIndex);  
        String data = value.substring(firstDashIndex + 1);  
        if (sign.equals(Crypto.sign(data,  
            Play.secretKey.getBytes())) {  
            String sessionData = URLDecoder.decode(data, "utf-8");  
            Matcher matcher = sessionParser.matcher(sessionData);  
            while (matcher.find()) {  
                session.put(matcher.group(1), matcher.group(2));  
            }  
        }  
    }  
}
```

They loop through the elements in the session

`\x00username:louis\x00\x00username:admin\x00` becomes

`"\x00username:louis\x00"` => `session.put("username" , "louis")`

`"\x00username:admin\x00"` => `session.put("username" , "admin")`



Play Session Injection



```
public static class Session {  
    static Pattern sessionParser =  
        Pattern.compile("\\u0000(?:[^\u0000]*):(?:[^\u0000]*)\\u0000");  
    [...]  
  
    String value = cookie.value;  
    int firstDashIndex = value.indexOf("-");  
    if (firstDashIndex > -1) {  
        String sign = value.substring(0, firstDashIndex);  
        String data = value.substring(firstDashIndex + 1);  
        if (sign.equals(Crypto.sign(data,  
            Play.secretKey.getBytes())) {  
            String sessionData = URLDecoder.decode(data, "utf-8");  
            Matcher matcher = sessionParser.matcher(sessionData);  
            while (matcher.find()) {  
                session.put(matcher.group(1), matcher.group(2));  
            }  
        }  
    }  
}
```

They loop through the elements in the session

`\x00username:louis\x00\x00username:admin\x00` becomes

`"\x00username:louis\x00"` => `session.put("username" , "louis")`

`"\x00username:admin\x00"` => `session.put("username" , "admin")` (OVERWRITE)

CVE ANALYSIS



Why?

- Deliberate practice
- Learning new patterns
- Learning how to fix issues
- Find incomplete patches



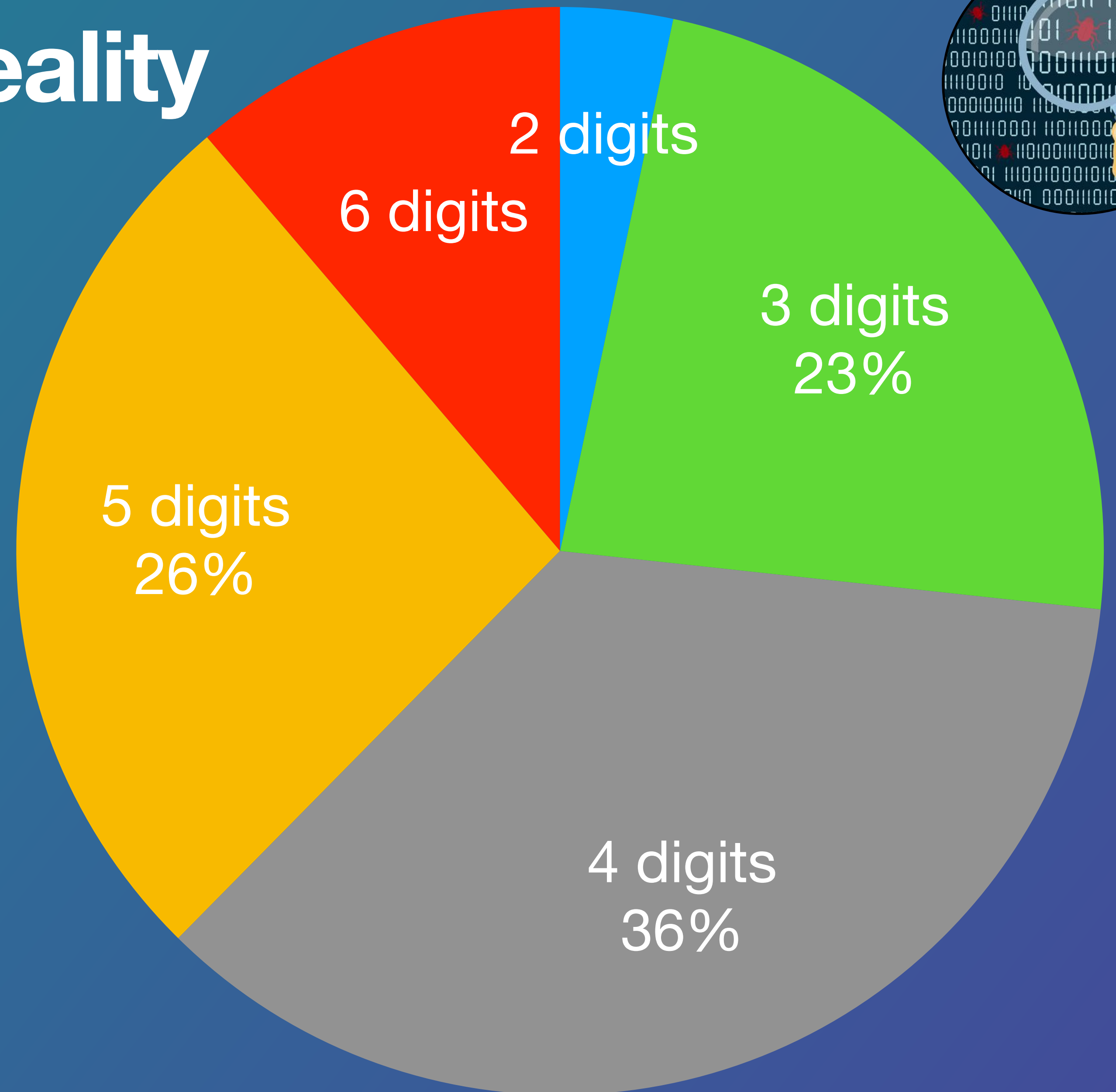
Analysing CVE (Code Review)

- Read the advisory!
- Clone the repository
- Find the tag for the vulnerable and fixed versions
- Extract a patch/diff
- Analyse the patch/diff:
 - What does the vulnerable code look like?
 - What does the fix look like?
 - Is this properly fixed?
 - More vulnerabilities in the same area?



Expectations vs Reality

575 CVEs (diff based on
vulnerable version versus
patched version):



Expectations vs Reality



[VULNERABLE]   [PATCHED]

[VULNERABLE]    [PATCHED]

[VULNERABLE]     [PATCHED]

[VULNERABLE]      [PATCHED]

[VULNERABLE]       [PATCHED]



Methodology



Analysing CVE (Code Review)

<https://github.com/zeromicro/go-zero/security/advisories/GHSA-fgxv-gw55-r5fq>

Authorization Bypass Through User-Controlled Key in go-zero

Critical kevwan published GHSA-fgxv-gw55-r5fq 2 weeks ago

Package

 github.com/zeromicro/go-zero (Go)

Affected versions

< v1.4.4

Patched versions

None

Severity

Critical 9.1 / 10



Analysing CVE (Code Review)

<https://github.com/zeromicro/go-zero/security/advisories/GHSA-fgxv-gw55-r5fq>



```
$ git clone https://github.com/zeromicro/go-zero/security/
```



Analysing CVE (Code Review)

<https://github.com/zeromicro/go-zero/security/advisories/GHSA-fgxv-gw55-r5fq>



```
$ git clone https://github.com/zeromicro/go-zero/
```

```
$ cd go-zero
```

```
$ git tag
```



Analysing CVE (Code Review)

<https://github.com/zeromicro/go-zero/security/advisories/GHSA-fgxv-gw55-r5fq>

Authorization Bypass Through User-Controlled Key in go-zero

Critical kevwan published GHSA-fgxv-gw55-r5fq 2 weeks ago

Package

 github.com/zeromicro/go-zero (Go)

Affected versions

< v1.4.4

Patched versions

None

Severity

Critical 9.1 / 10

```
$ cd go-zero
```

```
$ git tag
```

```
$ git diff v1.4.3...v1.4.4
```



```
$ git clone https://github.com/zeromicro/go-zero/
```

```
$ cd go-zero
```

```
$ git tag
```

```
$ git diff v1.4.3...v1.4.4
```

```
$ git diff v1.4.3...v1.4.4 | grep -i cors
```

```
$ git tag
```

```
$ git diff v1.4.3...v1.5.1 | grep -i cors
```



Analysing CVE (Code Review)

<https://github.com/zeromicro/go-zero/security/advisories/GHSA-fgxv-gw55-r5fq>



```
$ git diff v1.4.3...v1.5.1 | grep -i cors
@@ -535,3 +535,91 @@ func TestServer_WithCors(t *testing.T) {
snyff@snyffs-Air go-zero % git diff v1.4.3...v1.5.1 | grep cors
diff --git a/rest/internal/cors/handlers.go b/rest/internal/cors/handlers.go
--- a/rest/internal/cors/handlers.go
+++ b/rest/internal/cors/handlers.go
diff --git a/rest/internal/cors/handlers_test.go b/rest/internal/cors/handlers_test.go
--- a/rest/internal/cors/handlers_test.go
+++ b/rest/internal/cors/handlers_test.go
    "github.com/zeromicro/go-zero/rest/internal/cors"
```



```

% git diff v1.4.3...v1.5.1 rest/internal/cors/handlers.go
diff --git a/rest/internal/cors/handlers.go b/rest/internal/cors/handlers.go
index e2a64b74..133b47dd 100644
--- a/rest/internal/cors/handlers.go
+++ b/rest/internal/cors/handlers.go
@@ -77,12 +77,19 @@ func checkAndSetHeaders(w http.ResponseWriter, r *http.Request,
    origins []string
    }

    func isOriginAllowed(allows []string, origin string) bool {
-       for _, o := range allows {
-           if o == allOrigins {
+       origin = strings.ToLower(origin)
+
+       for _, allow := range allows {
+           if allow == allOrigins {
+               return true
+           }
+
+           allow = strings.ToLower(allow)
+           if origin == allow {
+               return true
+           }

-           if strings.HasSuffix(origin, o) {
+           if strings.HasSuffix(origin, "."+allow) {
+               return true
+           }
    }

```



```

% git diff v1.4.3...v1.5.1 rest/internal/cors/handlers.go
diff --git a/rest/internal/cors/handlers.go b/rest/internal/cors/handlers.go
index e2a64b74..133b47dd 100644
--- a/rest/internal/cors/handlers.go
+++ b/rest/internal/cors/handlers.go
@@ -77,12 +77,19 @@ func checkAndSetHeaders(w http.ResponseWriter, r *http.Request,
    origins []string
    }

    func isOriginAllowed(allows []string, origin string) bool {
-       for _, o := range allows {
-           if o == allOrigins {
+       origin = strings.ToLower(origin)
+       for _, allow := range allows {
+           if allow == allOrigins {
+               return true
+           }
+
+           allow = strings.ToLower(allow)
+           if origin == allow {
+               return true
+           }

-           if strings.HasSuffix(origin, o) {
+           if strings.HasSuffix(origin, "."+allow) {
                return true
            }
        }
    }

```

**They wanted to allow
an origin and all
subdomains of the
origin...**



```

% git diff v1.4.3...v1.5.1 rest/internal/cors/handlers.go
diff --git a/rest/internal/cors/handlers.go b/rest/internal/cors/handlers.go
index e2a64b74..133b47dd 100644
--- a/rest/internal/cors/handlers.go
+++ b/rest/internal/cors/handlers.go
@@ -77,12 +77,19 @@ func checkAndSetHeaders(w http.ResponseWriter, r *http.Request,
    origins []string
    }

    func isOriginAllowed(allows []string, origin string) bool {
-       for _, o := range allows {
-           if o == allOrigins {
+       origin = strings.ToLower(origin)
+
+       for _, allow := range allows {
+           if allow == allOrigins {
+               return true
+           }
+
+           allow = strings.ToLower(allow)
+           if origin == allow {
+               return true
+           }
+
+           if strings.HasSuffix(origin, o) {
+               if strings.HasSuffix(origin, "."+allow) {
+                   return true
                }
            }
        }
    }

```

They actually allowed all hostnames ending with the origin



```
% git diff v1.4.3...v1.5.1 rest/internal/cors/handlers.go
diff --git a/rest/internal/cors/handlers.go b/rest/internal/cors/handlers.go
index e2a64b74..133b47dd 100644
--- a/rest/internal/cors/handlers.go
+++ b/rest/internal/cors/handlers.go
@@ -77,12 +77,19 @@ func checkAndSetHeaders(w http.ResponseWriter, r *http.Request,
origins []string
}
```

```
func isOriginAllowed(allows []string, origin string) bool {
-   for _, o := range allows {
-       if o == allOrigins {
+   origin = strings.ToLower(origin)
+   for _, allow := range allows {
+       if allow == allOrigins {
+           return true
+       }
+       allow = strings.ToLower(allow)
+       if origin == allow {
+           return true
+       }
}
```

```
-   if strings.HasSuffix(origin, o) {
+   if strings.HasSuffix(origin, "."+allow) {
+       return true
+   }
}
```

**They actually allowed all
hostnames ending with
the origin**

pentesterlab.com ->
hackedbypentesterlab.com



Analysing CVE (Code Review)



```
$ git show v1.4.3:rest/internal/cors/handlers.go > handlers.go-before
```



Analysing CVE (Code Review)



```
$ git show v1.4.3:rest/internal/cors/handlers.go > handlers.go-before
```

```
$ git show v1.5.1:rest/internal/cors/handlers.go > handlers.go-after
```



JWT



The Compact JWS Format

3 Parts Separated by a dot

```
eyJ0eXAiOiJK  
V1QiLCJhbGci  
OiJIUzI1NiJ9
```

.

```
eyJsb2dpcbi  
I6ImFkb  
WluIn0
```

.

```
FSfvCBAwypJ4abF6  
jFLmR7JgZhkW674  
Z8dIdAIRyt1E
```

`eyJ = Base64('{"')`



The Compact JWS Format

Header and Payload are base64* encoded JSON

* urlsafe base64 encoding without padding

Base64 ({ ... })

■

Base64 ({ ... })

■

Base64 (...)

The signature is also base64 encoded



The Compact JWS Format

The header contains an algorithm “alg” attribute:

```
Base64 ( { "alg": "HS256",  
           "typ": "JWT" } )
```

·

...

·

...

To tell how the token was signed.

In this example HMAC with SHA256 was used



The JWT Format: header

The header contains an algorithm “alg” attribute:

```
Base64 ( { "alg": "HS256",  
           "typ": "JWT" } )
```

...

...

To tell how the token was signed.

In this example HMAC with SHA256 was used



The JWT Format: algorithm

A lot of different algorithms are supported*:

✓ None	✓ RS256	✓ ES256	✓ PS256
✓ HS256	✓ RS384	✓ ES384	✓ PS384
✓ HS384	✓ RS512	✓ ES512	✓ PS512
✓ HS512			

* <https://jwt.io/> covers most



Verifying a token

- Split the token in three parts based on the dots
- Base64 decode each part
- Parse the JSON for the header and payload
- Retrieve the algorithm from the header
- Verify the signature based on the algorithm

**The
Danger
Zone!**



Symmetric Signature



Sign!



Verify!



Secret



Asymmetric Signature



Sign!



Verify!



Private



Public



JWT: Algorithm Confusion

- JWT supports multiple signature mechanisms, such as:
 - RSA/EC
 - HMAC
- What happens if the application expects a signature using RSA or EC, but you provide a signature using HMAC?



JWT: Algorithm Confusion

- We are using RSA or EC, we verify the token using the public key

```
ALG.verify(PUBLIC_KEY)
```

Code written by the developers



JWT: Algorithm Confusion

- We are using RSA or EC, we verify the token using the public key

```
RSA.verify(PUBLIC_KEY)
```

Code written by the developers



JWT: Algorithm Confusion

- We are using RSA or EC, we verify the token using the public key

```
ALG.verify(PUBLIC_KEY)
```

User-Controlled header["alg"]

Code written by the developers



JWT: Algorithm Confusion

- We are using RSA or EC, we verify the token using the public key

HMAC.verify(PUBLIC_KEY)

Code written by the developers



How Libraries Prevent Algorithm Confusion Attacks?



Why is it interesting?

- The better you understand how attacks are blocked, the faster you can dismiss hypotheses during code review.
- The better you understand how attacks are blocked, the faster you'll recognise when something is worth investigating.
- Deliberate Practice!



Strategy #0: Only supporting HMAC

- No RSA/ECDSA, no algorithm confusion...
- A few libraries only support HMAC-based algorithms (such as <https://github.com/Corviz/jwt>)
- <https://github.com/brianvoe/sjwt> supports only HMAC SHA256



Strategy #1: Matching the alg header

The library requires developers to specify an algorithm and matches it with the alg header

```
def verify(jws, algorithm, key = nil)
  validate_alg_match(jws, algorithm)
  [...]

def validate_alg_match(jws, algorithm)
  header = decoded_header_json_to_hash(jws)
  unless alg_parameter(header) == algorithm
    fail("Algorithm not matching 'alg' header parameter")
  end
```



Strategy #2: Not relying on the alg header

The library requires developers to specify an algorithm and ignores the alg header

```
public static function validate(  
    TokenEncoded $tokenEncoded,  
    string $key,  
    string $algorithm, ?int $leeway = null,  
    ?array $claimsExclusions = null): bool  
{  
    $tokenDecoded = self::decode($tokenEncoded);  
  
    $signature = Base64Url::decode($tokenEncoded->getSignature());  
    $payload = $tokenDecoded->getPayload();  
  
    list($function, $type) = self::getAlgorithmData($algorithm);  
  
    switch ($function) {  
        case 'hash_hmac':  
            if (hash_equals(hash_hmac($type, $tokenEncoded->getMessage(), $key, true), $signature) !== true) {  
                throw new IntegrityViolationException('Invalid signature');  
            }  
            break;
```

[...]



Strategy #3: Strategy #1 + Strategy #2

The library requires developers to specify an algorithm, validates the alg header and doesn't use it

```
public DecodedJWT verify(DecodedJWT jwt) throws JWTVerificationException {  
    verifyAlgorithm(jwt, jwt.getAlgorithm());  
    jwt.getAlgorithm().verify(jwt);  
    verifyClaims(jwt, expectedChecks);  
    return jwt;  
}  
  
private void verifyAlgorithm(DecodedJWT jwt, Algorithm expectedAlgorithm) throws AlgorithmMismatchException {  
    if (!expectedAlgorithm.getName().equals(jwt.getAlgorithm())) {  
        throw new AlgorithmMismatchException(  
            "The provided Algorithm doesn't match the one defined in the JWT's Header.");  
    }  
}
```

Strategy #4: Preventing the use of public keys

Trying to match public keys when using HMAC

```
class HMACAlgorithm(Algorithm):  
    [...]  
  
    def prepare_key(self, key):  
        key = force_bytes(key)  
  
        if is_pem_format(key) or is_ssh_key(key):  
            raise InvalidKeyError(  
                "The specified key is an asymmetric key or x509 certificate and"  
                " should not be used as an HMAC secret."  
            )  
  
        return key
```



Strategy #4: Preventing the used of public keys

Trying to match public keys when using HMA

```
_PEM_RE = re.compile(
    b"----[- ]BEGIN ("
    + b"|".join(_PEMS)
    + b"")[- ]----\r?
    .+?\r?
    ----[- ]END \\1[- ]----\r?\n?"",
    re.DOTALL,
)

def is_pem_format(key: bytes) -> bool:
    return bool(_PEM_RE.search(key))d not be used as an HMAC secret."
    )

    return key
```

RISKY!



Strategy #4: Preventing the used of public keys

PyJWT took a while to cover all possible public keys:
See CVE-2022-29217 for an example.

PyJWT ended up requesting developers to provide one or more algorithms when verifying a token.

Also, what if my HMAC secret contains "BEGIN RSA PUBLIC KEY"?



Strategy #5: Verifying the type of the secret/key

<https://github.com/WebdevCave/jwt-php/>

```
abstract class Signer
{
    public function setSecret(Secret $secret): void
    {
        if (!$this->validateSecret($secret)) {
            throw new InvalidArgumentException('Invalid secret provided');
        }

        $this->secret = $secret;
    }
}
```



Strategy #5: Verifying the type of the secret/key

<https://github.com/WebdevCave/jwt-php/>



```
abstract class HsSigner extends Signer  
[...]
```

```
    protected function validateSecret(Secret $secret): bool  
{  
    return $secret instanceof HsSecret;  
}
```



Strategy #5: Verifying the type of the secret/key

https://bitbucket.org/b_c/jose4j/wiki/Home

```
public class HmacUsingShaAlgorithm extends AlgorithmInfo implements JsonWebSignatureAlgorithm
{
    [...]
    public boolean verifySignature(byte[] signatureBytes, Key key, byte[] securedInputBytes, ProviderContext
providerContext) throws JoseException
    {
        if (!(key instanceof SecretKey))
        {
            throw new InvalidKeyException(key.getClass() + " cannot be used for HMAC verification.");
        }

        Mac mac = getMacInstance(key, providerContext);
        byte[] calculatedSignature = mac.doFinal(securedInputBytes);

        return ByteUtil.secureEquals(signatureBytes, calculatedSignature);
    }
}
```



Strategy #6: Auto detection based on variable's type

<https://github.com/nov/json-jwt>

(but only for signing tokens, not to verify tokens)

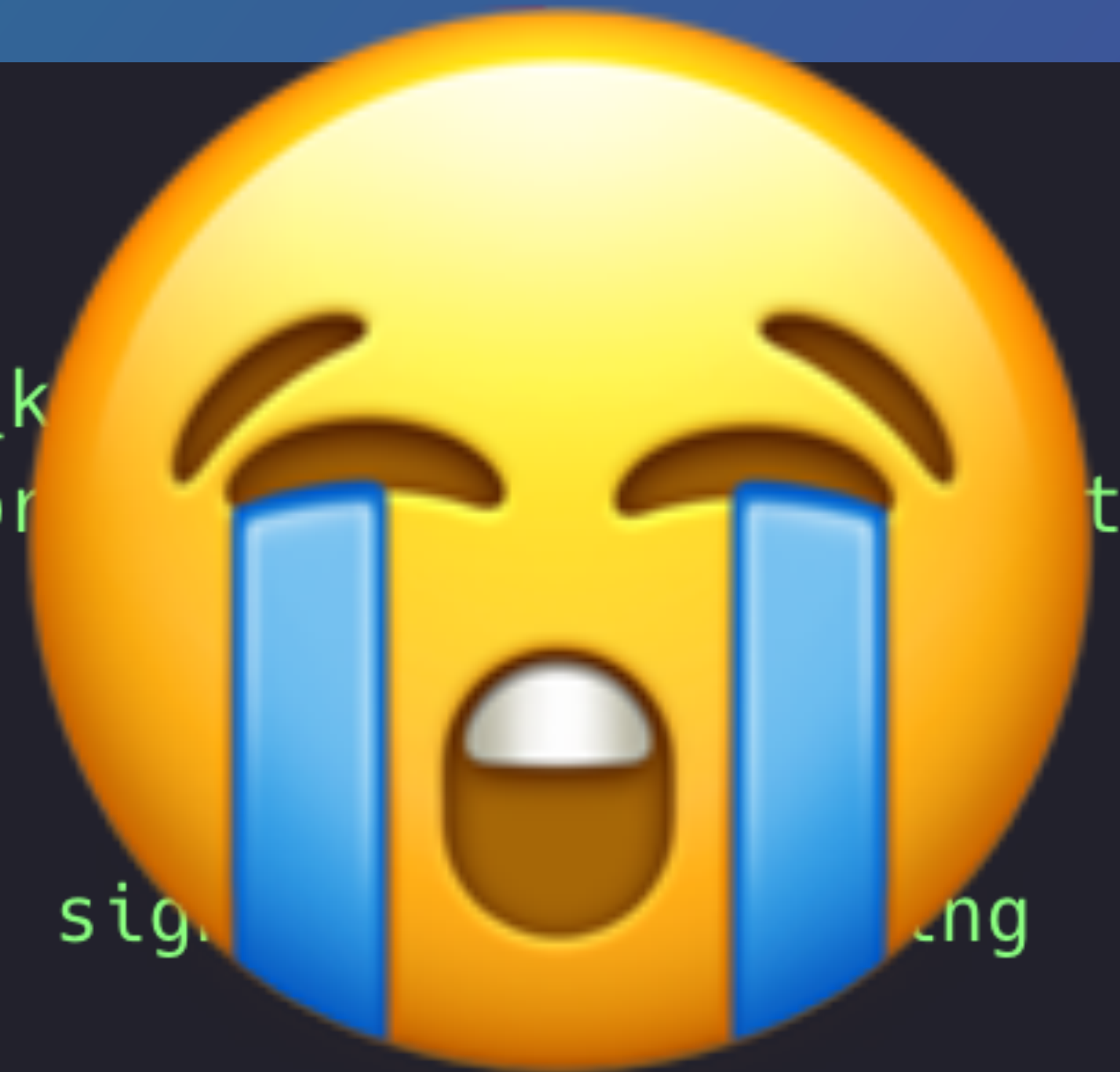
```
def autodetected_algorithm_from(private_key_or_secret)
  private_key_or_secret = with_jwk_support private_key_or_secret
  case private_key_or_secret
  when String
    :HS256
  when OpenSSL::PKey::RSA
    :RS256
  when OpenSSL::PKey::EC
    case private_key_or_secret.group.curve_name
    when 'prime256v1'
      :ES256
    when 'secp384r1'
      :ES384
    when 'secp521r1'
      :ES512
```

Strategy #7: Luck?

The code is almost vulnerable!

The call to `OpenSSL::HMAC.digest` will fail with a `TypeError` since there is no implicit conversion of `OpenSSL::PKey::RSA` to a `String`...

```
def sign(signature_base_string, private_key_or_secret)
  private_key_or_secret = with_jwk_support(private_key_or_secret)
  case
  when hmac?
    secret = private_key_or_secret
    OpenSSL::HMAC.digest digest, secret, signature_base_string
```



Let's see if analysing how libraries
prevent algorithm confusion
attacks helps us quickly discover
vulnerabilities...



Scala library



```
import org.janjaali.sprayjwt.Jwt
import org.janjaali.sprayjwt.algorithms.HS256
[...]  
val token = "..."  
val jsValue0pt = Jwt.decode(token, "super_fancy_secret")
```



Scala library



```
import org.janjaali.sprayjwt.Jwt
import org.janjaali.sprayjwt.algorithms.HS256
[...]  
val token = "..."  
val jsValue0pt = Jwt.decode(token, "super_fancy_secret")
```



Scala library

```
def decodeAsString(token: String, secret: String): Try[String] = {  
  val splitToken = token.split("\\.")  
  if (splitToken.length != 3) {  
    throw new InvalidJwtException("JWT must have form header.payload.signature")  
  }  
  
  val header = splitToken(0)  
  val payload = splitToken(1)  
  val data = s"$header.$payload"  
  
  val signature = splitToken(2)  
  
  val algorithm = getAlgorithmFromHeader(header)  
  
  if (!algorithm.validate(data, signature, secret)) {  
    throw new InvalidSignatureException()  
  }  
}
```



Scala library

```
def decodeAsString(token: String, secret: String): Try[String] = {  
  val splitToken = token.split("\\.")  
  if (splitToken.length != 3) {  
    throw new InvalidJwtException("JWT must have form header.payload.signature")  
  }  
  
  val header = splitToken(0)  
  val payload = splitToken(1)  
  val data = s"$header.$payload"  
  
  val signature = splitToken(2)  
  
  val algorithm = getAlgorithmFromHeader(header)  
  
  if (!algorithm.validate(data, signature, secret)) {  
    throw new InvalidSignatureException()  
  }  
}
```

The library relies on the alg header



Scala library

```
override def validate(data: String, signature: String, secret: String): Boolean = {
  val key = getPublicKey(secret)

  val dataByteArray = ByteEncoder.getBytes(data)

  val rsaSignature = Signature.getInstance(cryptoAlgName, provider)
  rsaSignature.initVerify(key)
  rsaSignature.update(dataByteArray)
  rsaSignature.verify(Base64Decoder.decode(signature))
}

private def getPublicKey(str: String): PublicKey = {
  val pemParser = new PEMParser(new StringReader(str))
  val keyPair = pemParser.readObject()

  Option(keyPair) match {
    case Some(publicKeyInfo: SubjectPublicKeyInfo) =>
      val converter = new JcaPEMKeyConverter
      converter.getPublicKey(publicKeyInfo)
    case _ => throw new IOException(s"Invalid key for $cryptoAlgName")
  }
}
```

Scala library

```
override def validate(data: String, signature: String, secret: String) Boolean = {  
  val key = getPublicKey(secret)  
  
  val dataByteArray = ByteEncoder.getBytes(data)  
  
  val rsaSignature = Signature.getInstance("RSA", "BC")  
  rsaSignature.initVerify(key)  
  rsaSignature.update(dataByteArray)  
  rsaSignature.verify(Base64Decoder.decode(signature))  
}  
  
private def getPublicKey(str: String): PublicKey = {  
  val pemParser = new PEMParser(new StringReader(str))  
  val keyPair = pemParser.readObject()  
  
  Option(keyPair) match {  
    case Some(publicKeyInfo: SubjectPublicKeyInfo) =>  
      val converter = new JcaPEMKeyConverter  
      converter.getPublicKey(publicKeyInfo)  
    case _ => throw new IOException(s"Invalid key for $cryptoAlgName")  
  }  
}
```

The public key used to verify the signature with RSA is provided as a String...

Scala library

Security Issue #5



Open

snyff opened this issue 3 weeks ago · 0 comments



snyff commented 3 weeks ago



Hi,

I hope this message finds you well. I've discovered a vulnerability in your library, and while it seems the project may not be actively maintained, I wanted to reach out to give you the opportunity to address the issue before I include it in one of my courses on code review and web hacking.

If you need more time to fix the issue, I can certainly wait longer. However, if I don't hear back from you within 30 days, I will assume you're not interested in resolving it and will proceed to publish the details.

Please let me know how I can share more information with you.

Best regards,
Louis



<https://github.com/xmidt-org/cjwt>

- I started looking at a few C libraries...
Including <https://github.com/xmidt-org/cjwt>
- Code from the examples/basic/rs_example.c

```
const char *rs_pub_key =  
    "-----BEGIN PUBLIC KEY-----\n"  
    ...  
    "V6L11BWkpzGXS4Hv43qa+GSY0D2QU68Mb59oSk20B+Bt0LpJofmbGEGgvmwyCI9\n"  
    "MwIDAQAB\n"  
    "-----END PUBLIC KEY-----";  
  
rv = cjwt_decode(rs_text, strlen(rs_text), 0,  
    (uint8_t *) rs_pub_key, strlen(rs_pub_key), 0, 0, &jwt);
```



<https://github.com/xmidt-org/cjwt>

The public key is provided as a string

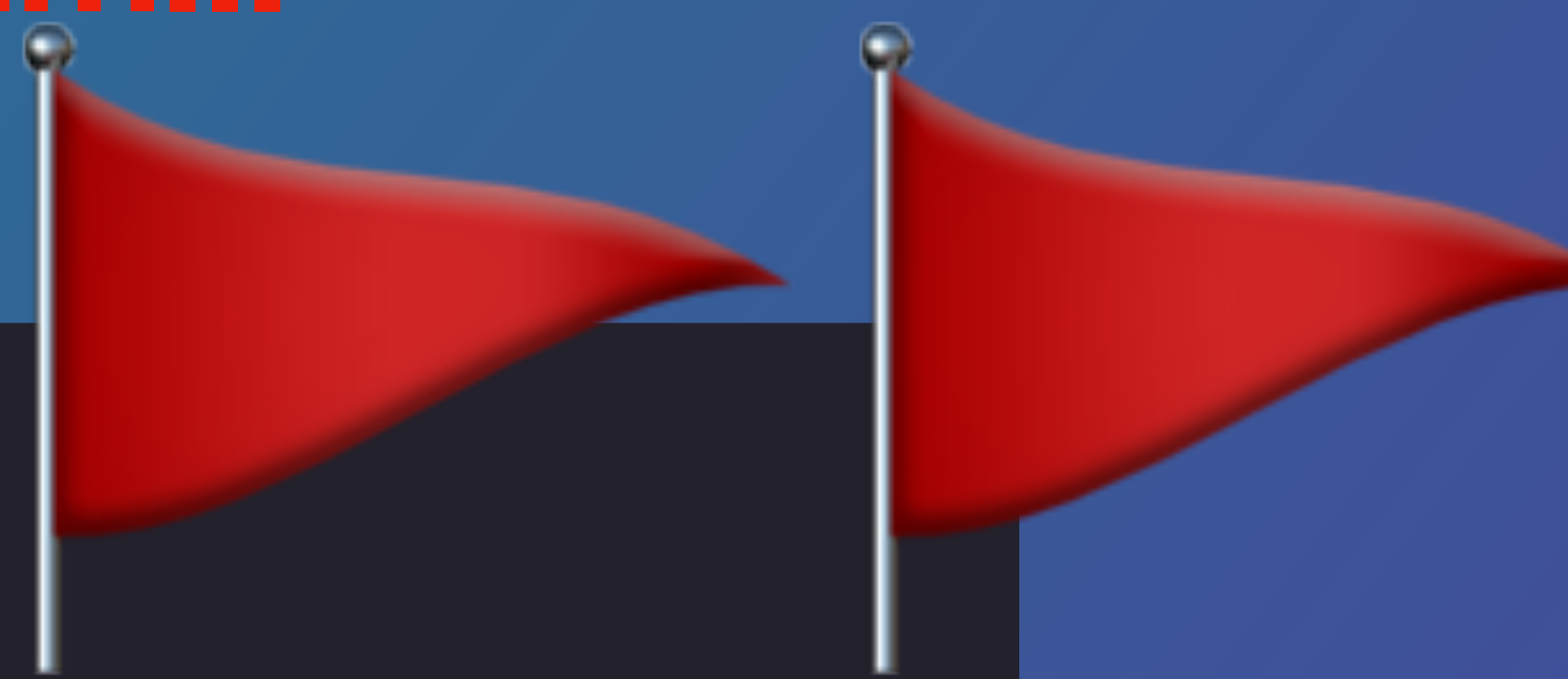


```
const char *rs_pub_key =  
    "-----BEGIN PUBLIC KEY-----\n"  
    ...  
    "V6L11BWkpzGXS4Hv43qa+GSY0D2QU68Mb59oSk20B+Bt0LpJofmbGEGgvmwyCI9\n"  
    "MwIDAQAB\n"  
    "-----END PUBLIC KEY-----";  
  
rv = cjwt_decode(rs_text, strlen(rs_text), 0,  
    (uint8_t *) rs_pub_key, strlen(rs_pub_key), 0, 0, &jwt);
```



<https://github.com/xmidt-org/cjwt>

Developers using the library
don't need to pick an algorithm...



```
const char *rs_pub_key =  
    "-----BEGIN PUBLIC KEY-----\n"  
    ...  
    "V6L11BWkpzGXS4Hv43qa+GSY0D2QU68Mb59oSk20B+Bt0LpJofmbGEGgvmwyCI9\n"  
    "MwIDAQAB\n"  
    "-----END PUBLIC KEY-----";  
  
rv = cjwt_decode(rs_text, strlen(rs_text), 0,  
    (uint8_t *) rs_pub_key, strlen(rs_pub_key), 0, 0, &jwt);
```



<https://github.com/xmidt-org/cjwt>

The code in the HMAC example is exactly the same as the RSA example



```
const char *hs_key = "hs256-secret";

rv = cjwt_decode(hs_text, strlen(hs_text), 0,
                (uint8_t *) hs_key, strlen(hs_key), 0, 0, &jwt);
```



<https://github.com/xmidt-org/cjwt>

```
cjwt_code_t jws_verify_signature(const cjwt_t *jwt, const struct sig_input *in)
{
    switch (jwt->header.alg) {
        case alg_es256:
            return verify_most(EVP_sha256(), in, EVP_PKEY_EC, 0);

        // [...]

        case alg_hs256:
            return verify_hmac(EVP_sha256(), in);
    }
}
```



<https://github.com/xmidt-org/cjwt>

- Reported on the 4th of December
- Fixed on the 18th of December
- Advisory published on the 19th of December



Conclusion (JWT)

- Enforcing the algorithm and relying on types are clear winning strategies to prevent algorithm confusion.
- Reviewing multiple libraries that offer the same functionality allows you to learn a great deal.
- Reviewing multiple libraries that offer the same functionality allows you to quickly detect weaknesses.



CONCLUSION 🧐



Assumptions!

Developers, yours, ...





**If you create software, you
may have as many CVEs as you want**



**And don't forget to
add them all to your CV!**



Keeping in touch



<https://pentesterlab.com/>



<https://pentesterlab.com/live-training/>



louis@pentesterlab.com



[@pentesterlab.com](#) and [@snyff.pentesterlab.com](#)



<https://www.linkedin.com/company/pentesterlab/>



<https://www.linkedin.com/in/snyff/>



[@PentesterLab](#) and [@snyff](#)



Conclusion

- Practice makes perfect
- There are still ****PLENTY**** of bugs to be found
- Keep notes!
- Now it's time to review some code!

